Operating and Maintaining the Urban Data Platform
Group 11

1 Generating Incremental Update Datasets

To simulate new data releases, we created three `data_generation` notebooks, one for each dataset to be extended. All generation notebooks are located in the `data_generation` directory. The generated datasets follow these common rules:

- New observations begin in January 2025, introducing future data into each dataset.
- Each dataset is extended by approximately 7%.
- New records preserve the original schema and format, except for intentional schema-evolution changes, and use realistic values within the historical data ranges to avoid unrealistic changes to the statistics.

1.1 Taxi Trips

For taxi trips, we used a subset of the publicly available January 2025 Yellow Taxi data rather than generating records from scratch. The 2025 source was reduced to 7% of the original dataset size. To test duplicate detection during incremental updates, we appended a further 1% of records from the original March 2024 taxi dataset.

1.2 Weather

The weather dataset was extended with the first 7% of the original 2024 observations. Their year was changed to 2025, and selected weather variables were modified by small random amounts while keeping nearby observations similar. For example, temperature received a random shift centered around the original value. All modified values were clipped to the minimum and maximum values of the original dataset. A new `humidity` column was generated using uniformly distributed values between 20% and 100%.

1.3 Air Quality

For air quality, we copied the earliest complete GMT days representing approximately 7% of the original observations. The GMT and local timestamps were shifted forward so that the generated data begins immediately after the latest source observation, covering January 2025. The original measurement values were kept unchanged, while a new `aqi` column was generated with values between 0 and 500. The `Date of Last Change` and local time fields were also updated to match the shifted dates.

1.4 Incremental Update Pipeline

The update pipeline appends new source records to the Bronze layer and incrementally processes them in the Silver layer. Bronze records include ingestion metadata, and Silver uses this metadata to identify records that have not yet been processed.

1.5 Schema Evolution

To handle schema evolution, the framework uses a version-isolated configuration strategy based on JSON configuration files.

- Each Bronze record is labeled with a `schema_version` from its configuration file.

- Silver filters Bronze records by schema version, then validates and transforms them using the matching configuration before writing them to the Silver layer.
- Delta schema merging is enabled when writing Bronze and Silver data, allowing compatible added columns. Values for a column unavailable in a particular version are stored as `NULL`. However, a required missing input column causes the batch to be rejected.
- Version-specific `rename_columns` and `cast_columns` operations can standardize differing column names or types after the incoming data has passed its configured schema validation.

2 Maintain Analytical Consistency

The platform uses different recomputation strategies to reduce unnecessary analytical processing after an update. Each data product defines a `recompute_mode` and merge keys. For `incremental` mode, the product query runs only on Gold executions that have not previously been included in that product, and the result is appended. For `full` mode, the product is recalculated from the full source table and merged using its configured keys.

2.1 Discussion

- **Which analytical data products can be refreshed incrementally?**
  - monthly_zone_demand
  - air_quality_vs_demand
- **Which require complete recomputation?**
  - `average_trip_distance_by_weather`, because an average for each weather condition depends on all historical trips.
  - `zone_weather_demand_variation`, because its variation metric depends on the complete demand distribution for each zone.
- **Which schema changes can be handled automatically?** Another aspect of the new framework, that it is able to handle the evolution of the data products in terms of extensions. For any new column or logic, the framework can use the new versions of the definition without breaking the old products when the changes only include changes to calculations or extensions with new columns.
- **Which schema changes require manual intervention?** If the new schema requires columns to be dropped, for the old data product the drop operation has to be executed manually.
- **How does your design reduce unnecessary computation?** This method allows for cheap extension of data products with incremental data, while keeping the process idempotent. It is important to mention that even if recomputation happens and new data arrives, only modifications will be written into delta, and overwrites with identical values are avoided.

3 Build a Platform Monitoring System

3.1 Monitoring Design

The platform records one monitoring entry for every execution in the Bronze, Silver, Gold, and analytical-product refresh stages. These entries are stored in the `metadata/pipeline_executions` Delta table and capture all the necessary metadata.

3.2 Metadata Model

The `pipeline_executions` table stores the pipeline and execution identifiers, stage, dataset, schema version, start and end timestamps, duration, processed, inserted, and rejected records, and validation-failure counts. A `pipeline_run_id` identifies a complete pipeline run, while each Bronze, Silver, Gold, or analytical-product job receives its own `execution_id`. After each job completes, its metadata is appended to the table, preserving execution history for monitoring and comparison.

3.3 Monitoring Queries

The monitoring queries were executed on the `pipeline_executions` table using `platform_monitoring_queries.ipynb`.

- **Which dataset fails validation most frequently?** `taxi_trips` had two failing executions, with 306,939 rejected records and 919,779 rule violations in total. `taxi_zones` had one rejected record, while weather and air quality had no rejections.
- **Which dataset requires the longest processing time?** Based on average execution duration, Silver processing of `air_quality` took the longest, averaging 45.523 seconds. Its longest individual execution took 61.423 seconds.
- **How many records were rejected during each execution?** Rejections occurred only during Silver processing: the `taxi_trips` full-ingestion pipeline rejected 194,717 records, while the incremental-update pipeline rejected 112,222 records. `taxi_zones` rejected one record.
- **How has processing time changed over multiple executions?** Processing times differed between the full-ingestion and incremental-update pipelines. Bronze and Gold processing became faster during the update. For example, Gold processing decreased from 35.411 to 15.734 seconds. However, Silver processing increased because incremental processing also performs cross-batch validation against existing Silver data.

3.4 Discussion

- **Which operational metrics are most useful?**
  - `duration_seconds`: Helps identify performance regressions.
  - `processed_records, inserted_records`: Reveal changes in input volume and rows successfully processed.
  - `rejected_records, validation_failure_count`: Expose data-quality issues. Comparing them as a rejection rate is especially useful across datasets of different sizes.
  - `stage, dataset, schema_version, execution_id, pipeline_run_id`: Provide the context needed to compare and trace executions.
- **How could this information support debugging and system maintenance?**
  - A spike in `rejected_records` or `validation_failure_count` identifies a data-quality problem in a specific dataset and execution.
  - The rejected-records Delta table can be inspected to view the raw invalid records and their rejection reasons.
  - Increasing `duration_seconds` over multiple runs identifies stages that may need optimization because of inefficient joins or growing data volumes.
  - `pipeline_run_id` and `execution_id` trace a problem across related Bronze, Silver, Gold, and analytical-refresh stages.
  - `schema_version` helps determine whether an issue was introduced by a schema change.
- **What additional monitoring information would be valuable in a production system?**

- Execution status and error messages, because a failed job may not create a normal completion record.
- Source-file names and arrival times to detect missing or delayed inputs.
- Resource usage, such as memory and CPU, to identify infrastructure bottlenecks.
- Alerts when validation failures or processing time exceed defined thresholds.

4 Extend the Data Validation Framework

4.1 Validation Design

1. Each dataset JSON config file defines quality-check actions and parameters. The Silver job passes these checks to `run_quality_checks`.

2. For schema-valid input, each row starts with an empty `rejection_reasons` array. Validators add a reason whenever a rule is violated, so a row can contain multiple failure reasons. Unsupported schema changes instead add schema-error reasons to every row in the affected batch.

3. After all checks run, rows with no rejection reasons continue to transformation and Silver. Rows with one or more reasons are written to the Delta table at `metadata/rejected_records`. Thus, invalid records do not interrupt the pipeline or reach Silver, Gold, or analytical products.

4.2 Supported Validation Rules

- `validate_primary_key`: Rejects missing or duplicate key values.
- `validate_null_values`: Rejects null values and configured missing-value markers.
- `validate_allowed_values`: Rejects values outside a configured set.
- `validate_non_negative_numbers`: Rejects negative values in numeric columns.
- `validate_numeric_range`: Rejects values outside configured minimum and maximum limits.
- `validate_duplicates`: Rejects duplicate rows based on selected columns.
- `validate_reference_exists`: Rejects values with no matching record in a referenced Silver dataset.

4.3 Schema Validation and Versioning

Bronze data is validated against the active configuration schema before quality checks. Missing columns, incompatible types, and populated unexpected columns reject the batch.

Records are filtered by `schema_version`, so each configuration processes only matching Bronze records. During incremental updates, a dataset-level watermark based on the latest Silver execution selects newly ingested Bronze records. Null-only fields retained by Delta from other schema versions are ignored during schema validation.

4.4 Rejected-Records

Rejected rows are stored in the Delta table `metadata/rejected_records`. Each record includes pipeline and execution IDs, stage, dataset, schema version, rejection timestamp, rejection reasons, and the raw source record.

Quarantined rows are excluded from Silver, Gold, and analytical products.

4.5 Discussion

- **Which validation rules are generic and can be applied across all datasets?** All validator implementations are generic and reusable, but their applicability depends on the dataset. Schema, null-value, and duplicate checks are broadly useful, while primary-key, numeric-range, allowed-value, and reference-existence checks require dataset-specific columns, business rules, or relationships.
- **Which validation rules are dataset-specific?** No validator implementation is dataset-specific. Dataset-specific behavior is defined through configuration parameters such as columns, allowed values, numeric ranges, and reference datasets.
- **How can new validation rules be added without modifying the core validation framework?** Existing rule types are added through JSON configuration entries. A new validator type requires a validator function and one registration in `VALIDATOR_REGISTRY`, but the shared validation flow in `run_quality_checks` does not need to change.